

# Optimización y Análisis de Redes

---

Guía de la  
Asignatura

Grado en Matemáticas

## AUTORES

- Víctor Aceña Gil
- Antonio Alonso Ayuso

2026-2027



# Índice de contenidos

|          |                                                                                       |           |
|----------|---------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introducción a la asignatura</b>                                                   | <b>2</b>  |
| <b>2</b> | <b>Datos identificativos y competencias</b>                                           | <b>3</b>  |
| 2.1      | Ficha técnica de la asignatura . . . . .                                              | 3         |
| 2.2      | Competencias generales y específicas . . . . .                                        | 3         |
| <b>3</b> | <b>Metodología docente y distribución del trabajo</b>                                 | <b>4</b>  |
| <b>4</b> | <b>Cronograma y plan de trabajo semanal</b>                                           | <b>5</b>  |
| 4.1      | BLOQUE I: Optimización Matemática y Ciencia de Datos (Semanas 1 a 10) . . . . .       | 5         |
| 4.1.1    | SEMANA 1: Fundamentos y Modelado Matemático de la Optimización . . . . .              | 5         |
| 4.1.2    | SEMANA 2: Optimización No Lineal Sin Restricciones . . . . .                          | 6         |
| 4.1.3    | SEMANA 3: Algoritmos de Descenso y Visualización Avanzada . . . . .                   | 6         |
| 4.1.4    | SEMANA 4: Análisis Convexo I - Conjuntos Convexos . . . . .                           | 7         |
| 4.1.5    | SEMANA 5: Análisis Convexo II - Funciones Convexas . . . . .                          | 7         |
| 4.1.6    | SEMANA 6: Optimización con Restricciones y Condiciones KKT . . . . .                  | 8         |
| 4.1.7    | SEMANA 7: Dualidad Lagrangiana . . . . .                                              | 8         |
| 4.1.8    | SEMANA 8: Programación Convexa Disciplinada y CVXPY . . . . .                         | 9         |
| 4.1.9    | SEMANA 9: Aplicaciones a Ciencia de Datos I - Regresión y SVM Lineal . . . . .        | 9         |
| 4.1.10   | SEMANA 10: Aplicaciones a Ciencia de Datos II - SVM Generalizada y Dualidad . . . . . | 10        |
| 4.2      | BLOQUE II: Optimización y Análisis de Redes (Semanas 11 a 15) . . . . .               | 10        |
| 4.2.1    | SEMANA 11: Teoría de Grafos, Estructuras y Árboles Soporte . . . . .                  | 10        |
| 4.2.2    | SEMANA 12: Algoritmos de Caminos Mínimos . . . . .                                    | 11        |
| 4.2.3    | SEMANA 13: Problemas de Flujo en Redes . . . . .                                      | 11        |
| 4.2.4    | SEMANA 14: Problemas de Emparejamiento (Matching) y Asignación . . . . .              | 12        |
| 4.2.5    | SEMANA 15: Rutas en Redes, TSP, VRP y Síntesis Final . . . . .                        | 12        |
| <b>5</b> | <b>Sistema de evaluación y criterios de calificación</b>                              | <b>14</b> |
| 5.1      | Convocatoria Ordinaria (Evaluación Continua) . . . . .                                | 14        |
| 5.2      | Convocatoria Extraordinaria . . . . .                                                 | 14        |
| <b>6</b> | <b>Bibliografía y recursos de referencia</b>                                          | <b>15</b> |
| 6.1      | Textos de referencia principal . . . . .                                              | 15        |
| 6.2      | Textos complementarios de algoritmos y computación . . . . .                          | 15        |
| 6.3      | Recursos digitales y software . . . . .                                               | 15        |

# 1 Introducción a la asignatura

La **optimización matemática** y el **análisis de redes** constituyen dos de los pilares fundamentales de la matemática aplicada, la investigación operativa y la ciencia de datos moderna. En la sociedad digital e industrial contemporánea, donde los recursos son finitos y los datos interconectados crecen de forma exponencial, la capacidad de formalizar matemáticamente problemas complejos y resolverlos algorítmicamente resulta esencial para la toma de decisiones estratégicas.

Esta asignatura, diseñada para el **Grado en Matemáticas** de la **Universidad Rey Juan Carlos (URJC)**, amplía y consolida la formación matemática previa en álgebra lineal, cálculo multivariante y programación lineal, adentrándose en el análisis riguroso de la **optimización no lineal continua** y el modelado discreto mediante la **teoría y optimización en redes**.

El diseño pedagógico del curso conjuga el rigor deductivo formal (teoremas de existencia, convexidad, dualidad lagrangiana y complejidad algorítmica) con el desarrollo computacional en **Python**, empleando librerías científicas profesionales de referencia internacional como **NumPy**, **SciPy**, **CVXPY** (para optimización convexa disciplinada) y **NetworkX** (para el modelado y visualización de grafos).

---

## 2 Datos identificativos y competencias

### 2.1. Ficha técnica de la asignatura

- **Asignatura:** Optimización y Análisis de Redes
- **Titulación:** Grado en Matemáticas / Dobles Grados afines
- **Centro:** Escuela Técnica Superior de Ingeniería Informática (ETSII), Universidad Rey Juan Carlos
- **Créditos ECTS:** 6.0 ECTS (150 horas totales de dedicación del estudiante)
- **Carácter:** Formación de especialidad / Obligatoria de rama
- **Temporalidad:** Segundo cuatrimestre (15 semanas lectivas)
- **Idioma de impartición:** Castellano
- **Prerrequisitos recomendados:** Álgebra Lineal, Cálculo Diferencial e Integral en Varias Variables, Fundamentos de Programación e Introducción a la Optimización Lineal.

### 2.2. Competencias generales y específicas

Al superar la asignatura, los estudiantes habrán adquirido las siguientes capacidades formativas:

1. **Modelización matemática formal:** Capacidad para abstraer problemas reales de logística, telecomunicaciones, finanzas y ciencia de datos, formulándolos rigurosamente como problemas de optimización matemática o modelos sobre redes.
  2. **Análisis teórico y deductivo:** Habilidad para caracterizar la convexidad de conjuntos y funciones, demostrar condiciones de optimalidad de primer y segundo orden (KKT) y formular problemas duales interpretando sus propiedades de convergencia.
  3. **Algoritmia y complejidad computacional:** Comprensión profunda de la estructura, convergencia y coste asintótico de los algoritmos clásicos y modernos de optimización en espacios continuos y sobre grafos finitos.
  4. **Implementación y experimentación computacional:** Destreza para programar algoritmos numéricos y combinatorios en Python, validar soluciones con optimizadores comerciales y de código abierto, y comunicar resultados técnicos mediante visualizaciones claras.
-

### 3 Metodología docente y distribución del trabajo

La asignatura se articula en torno a un modelo de aprendizaje activo y equilibrado entre la fundamentación analítica y la experimentación computacional en laboratorio:

- **Clases de teoría y fundamentos matemáticos (2.0 horas/semana):** Deducción formal de teoremas, análisis poliédrico, demostración de propiedades de convexidad, condiciones de Karush-Kuhn-Tucker y dualidad.
- **Sesiones de laboratorio en Python (2.0 horas/semana):** Implementación práctica de algoritmos, resolución de modelos numéricos con CVXPY y NetworkX, y visualización de soluciones y superficies de nivel con Matplotlib.
- **Seminarios y resolución guiada de problemas:** Análisis paso a paso de casos de estudio, formulaciones complejas de programación entera y ejercicios de examen.
- **Tutorías académicas personalizadas:** Orientación individual y en grupo para la resolución de dudas teóricas y seguimiento de los proyectos de laboratorio.
- **Trabajo autónomo del estudiante (6.0 horas/semana):** Estudio personal, lectura de la bibliografía recomendada, preparación de sesiones prácticas y resolución de colecciones de problemas.

| Actividad formativa                | Horas presenciales | Horas de trabajo autónomo | Horas totales  |
|------------------------------------|--------------------|---------------------------|----------------|
| Clases de teoría magistral         | 30.0 h             | 35.0 h                    | 65.0 h         |
| Prácticas de laboratorio en Python | 26.0 h             | 34.0 h                    | 60.0 h         |
| Seminarios y evaluación continua   | 4.0 h              | 21.0 h                    | 25.0 h         |
| <b>Total dedicación (6 ECTS)</b>   | <b>60.0 h</b>      | <b>90.0 h</b>             | <b>150.0 h</b> |

## 4 Cronograma y plan de trabajo semanal

El programa docente se estructura cronológicamente a lo largo de las 15 semanas lectivas del cuatrimestre, dividido en dos grandes bloques temáticos complementarios:

---

### 4.1. BLOQUE I: Optimización Matemática y Ciencia de Datos (Semanas 1 a 10)

#### 4.1.1. SEMANA 1: Fundamentos y Modelado Matemático de la Optimización

Contenidos teóricos (Tema 1):

- Naturaleza de la optimización: variables de decisión, parámetros, funciones objetivo y regiones factibles.
- Clasificación taxonómica: problemas continuos vs. discretos, lineales vs. no lineales, convexos vs. no convexos.
- Repaso estructurado de álgebra lineal numérica y cálculo diferencial multivariante.

Sesión de laboratorio en Python:

- Configuración del entorno científico (**Anaconda**, entornos virtuales **conda**, **VS Code** y **Jupyter**).
- Manipulación vectorial, cálculo matricial y álgebra lineal numérica con **NumPy**.

Objetivos de aprendizaje:

1. **Identificar** y **formular** matemáticamente los elementos constitutivos de un problema de optimización.
  2. **Manipular** matrices, gradientes y normas vectoriales de forma eficiente en Python.
-

#### 4.1.2. SEMANA 2: Optimización No Lineal Sin Restricciones

##### Contenidos teóricos (Tema 2):

- Caracterización analítica de óptimos locales y globales en  $\mathbb{R}^n$ .
- Condiciones necesarias de primer orden (puntos estacionarios).
- Condiciones necesarias y suficientes de segundo orden mediante matrices Hessianas.
- Concepto de dirección de descenso y velocidad de convergencia.

##### Sesión de laboratorio en Python:

- Programación modular del algoritmo de **Descenso de Gradiente (Gradient Descent)**.
- Análisis del efecto de la tasa de aprendizaje (*learning rate*) en funciones cuadráticas mal condicionadas.

##### Objetivos de aprendizaje:

1. **Calcular** y **clasificar** puntos críticos de funciones no lineales mediante su matriz Hessiana.
  2. **Implementar** el descenso de gradiente básico y evaluar su trayectoria en el espacio de parámetros.
- 

#### 4.1.3. SEMANA 3: Algoritmos de Descenso y Visualización Avanzada

##### Contenidos teóricos (Tema 2):

- Métodos de búsqueda lineal unidimensional: regla de Armijo y condiciones de Wolfe.
- El **Método de Newton**: convergencia cuadrática y coste computacional del cálculo e inversión del Hessiano.
- Introducción a métodos cuasi-Newton (BFGS) y gradiente conjugado.

##### Sesión de laboratorio en Python:

- Visualización de trayectorias de optimización y curvas de nivel en 2D y superficies 3D con **Matplotlib**.
- Uso profesional de solvers de optimización sin restricciones en **scipy.optimize** (**minimize**).

##### Objetivos de aprendizaje:

1. **Comparar** la convergencia lineal del gradiente frente a la convergencia cuadrática de Newton.
  2. **Generar** representaciones gráficas tridimensionales rigurosas de paisajes de coste y trayectorias de descenso.
-

#### 4.1.4. SEMANA 4: Análisis Convexo I - Conjuntos Convexos

##### Contenidos teóricos (Tema 3):

- Definición geométrica y algebraica de conjunto convexo y combinaciones convexas.
- Envoltura convexa (*convex hull*) y conos convexos.
- Familias canónicas: hiperplanos, semiespacios, poliedros, normas y elipsoides.
- Teoremas de separación de convexos e hiperplanos de soporte.
- Operaciones que preservan la convexidad de conjuntos (intersección, transformaciones afines).

##### Sesión de laboratorio en Python:

- Representación y verificación computacional de poliedros formados por intersección de semiespacios.
- Cálculo computacional de envolturas convexas en 2D y 3D utilizando `scipy.spatial`.

##### Objetivos de aprendizaje:

1. **Demostrar** rigurosamente la convexidad de conjuntos matemáticos parametrizados.
  2. **Aplicar** teoremas de separación para verificar la disyunción entre regiones convexas.
- 

#### 4.1.5. SEMANA 5: Análisis Convexo II - Funciones Convexas

##### Contenidos teóricos (Tema 3):

- Definición de función convexa, estrictamente convexa y fuertemente convexa.
- Caracterización mediante el epígrafe (*epigraph*) y conjuntos de subnivel.
- Condiciones diferenciales de primer orden (subgradientes y plano tangente inferior).
- Condiciones de segundo orden: semidefinición positiva de la matriz Hessiana ( $\nabla^2 f(x) \succeq 0$ ).
- Teorema fundamental: equivalencia entre óptimos locales y globales en dominios convexas.

##### Sesión de laboratorio en Python:

- Verificación numérica de la convexidad de funciones multivariantes mediante análisis espectral de Hessianos.
- Experimentación de la desigualdad de Jensen en distribuciones empíricas.

##### Objetivos de aprendizaje:

1. **Analizar** la convexidad de funciones multivariantes mediante criterios de primer y segundo orden.
  2. **Comprender** y **explicar** por qué la convexidad elimina el riesgo de quedar atrapado en óptimos locales espurios.
-



#### 4.1.6. SEMANA 6: Optimización con Restricciones y Condiciones KKT

##### Contenidos teóricos (Tema 4):

- Formulación general de problemas no lineales con restricciones de igualdad y desigualdad.
- Geometría de las restricciones y conos de direcciones factibles y de descenso.
- Cualificación de restricciones: condición de independencia lineal de gradientes (LICQ) y condición de Mangasarian-Fromovitz (MFCQ).
- Teorema de las **condiciones de Karush-Kuhn-Tucker (KKT)**: primal factibilidad, dual factibilidad, holgura complementaria y estacionariedad.
- Suficiencia de KKT bajo hipótesis de convexidad.

##### Sesión de laboratorio en Python:

- Resolución analítica y comprobación numérica de sistemas no lineales KKT.
- Visualización gráfica de conos de gradientes activos y multiplicadores de Lagrange en 2D.

##### Objetivos de aprendizaje:

1. **Formular** y **resolver** analíticamente las ecuaciones de KKT en problemas con restricciones no lineales.
  2. **Interpretar** el significado geométrico y físico de las restricciones activas e inactivas.
- 

#### 4.1.7. SEMANA 7: Dualidad Lagrangiana

##### Contenidos teóricos (Tema 4):

- Construcción formal de la función Lagrangiana y de la función dual de Lagrange  $g(\lambda, \mu)$ .
- Concavidad intrínseca de la función dual.
- El problema Dual de Lagrange y la propiedad de **Dualidad Débil** (salto dual o *duality gap*).
- Teoremas de **Dualidad Fuerte** y condiciones de cualificación de Slater para problemas convexos.
- Interpretación económica y análisis de sensibilidad de los multiplicadores duales (precios sombra).

##### Sesión de laboratorio en Python:

- Construcción numérica y trazado de la función dual  $g(\lambda)$  en problemas cuadráticos.
- Verificación empírica de la dualidad fuerte y convergencia primal-dual.

##### Objetivos de aprendizaje:

1. **Construir** la función dual de Lagrange y formular el problema dual asociado a un problema primal dado.
  2. **Evaluar** la existencia de dualidad fuerte mediante la condición de Slater.
-

#### 4.1.8. SEMANA 8: Programación Convexa Disciplinada y CVXPY

##### Contenidos teóricos (Tema 4):

- Jerarquía de problemas convexos: Programación Lineal (LP), Programación Cuadrática (QP), Programación Cuadrática con Restricciones Cuadráticas (QCQP) y Conos de Segundo Orden (SOCP).
- Principios de la **Programación Convexa Disciplinada (DCP)**: reglas de composición atomística y análisis de curvatura.

##### Sesión de laboratorio en Python:

- Introducción exhaustiva a la librería **CVXPY**.
- Formulación declarativa de variables, parámetros, restricciones y funciones objetivo.
- Resolución de problemas LP y QP, inspección del estado de los solvers y recuperación de variables primales y multiplicadores duales.

##### Objetivos de aprendizaje:

1. **Traducir** modelos matemáticos abstractos a código ejecutable en CVXPY respetando las reglas DCP.
  2. **Interpretar** los certificados de optimalidad o infactibilidad devueltos por el solver.
- 

#### 4.1.9. SEMANA 9: Aplicaciones a Ciencia de Datos I - Regresión y SVM Lineal

##### Contenidos teóricos (Tema 5):

- Formulación matricial de la regresión lineal por mínimos cuadrados ordinarios (OLS).
- Regularización y selección de variables: regresión Ridge ( $L_2$ ) y regresión Lasso ( $L_1$ ).
- Fundamentación del aprendizaje estadístico: clasificación binaria e hiperplanos separadores.
- **Máquinas de Vectores de Soporte (SVM)**: formulación matemática de margen máximo para datos linealmente separables.

##### Sesión de laboratorio en Python:

- Implementación y comparación de modelos de regresión OLS, Ridge y Lasso en CVXPY.
- Formulación y resolución del clasificador SVM lineal de margen duro en CVXPY.
- Visualización gráfica del hiperplano de separación, márgenes y vectores de soporte.

##### Objetivos de aprendizaje:

1. **Formular** problemas canónicos de aprendizaje automático como problemas de optimización convexa.
  2. **Calcular** e **ilustrar** los vectores de soporte críticos que definen la frontera de decisión.
-

#### 4.1.10. SEMANA 10: Aplicaciones a Ciencia de Datos II - SVM Generalizada y Dualidad

##### Contenidos teóricos (Tema 5):

- Clasificación con solapamiento: margen blando y variables de holgura  $\xi_i$ .
- Parámetro de regularización  $C$  y compromiso sesgo-varianza (*bias-variance tradeoff*).
- Formulación dual de SVM con variables multiplicadoras  $\alpha_i$  acotadas en  $[0, C]$ .
- El **truco del Kernel (Kernel Trick)**: funciones de base radial (RBF) y núcleos polinomiales en espacios de Hilbert de dimensión infinita.

##### Sesión de laboratorio en Python:

- Resolución del clasificador SVM dual no lineal en CVXPY.
- Comparativa de rendimiento y tiempo de cómputo frente a la implementación de `scikit-learn` (`sklearn.svm.SVC`).
- Evaluación cruzada y ajuste de hiperparámetros  $(C, \gamma)$ .

##### Objetivos de aprendizaje:

1. **Diferenciar** y **derivar** las formulaciones primal y dual de las Máquinas de Vectores de Soporte.
  2. **Aplicar** transformaciones por kernels para resolver problemas de clasificación no linealmente separables.
- 

## 4.2. BLOQUE II: Optimización y Análisis de Redes (Semanas 11 a 15)

### 4.2.1. SEMANA 11: Teoría de Grafos, Estructuras y Árboles Soporte

##### Contenidos teóricos (Tema 6):

- Elementos formales de teoría de grafos: grafos no dirigidos, digrafos, multigrafos y grafos bipartitos.
- Representaciones algebraicas: matriz de adyacencia, matriz de incidencia y matriz Laplaciana.
- Conectividad: caminos, componentes conexas, conjuntos de corte y árboles.
- **Árboles Soporte de Mínimo Peso (MST)**: algoritmo de Kruskal y algoritmo de Prim.

##### Sesión de laboratorio en Python:

- Modelado y manipulación de topologías de red con **NetworkX**.
- Cálculo de métricas estructurales (grados, densidades, componentes conexas).
- Implementación y comparación de algoritmos MST sobre grafos sintéticos y redes reales.

##### Objetivos de aprendizaje:

1. **Representar** infraestructuras de transporte y redes complejas mediante estructuras formales de grafos.

2. **Calcular** el árbol generador de coste mínimo de una red valorada aplicando algoritmos voraces (*greedy*).
- 

#### 4.2.2. SEMANA 12: Algoritmos de Caminos Mínimos

Contenidos teóricos (Tema 7):

- El Problema del Camino Mínimo (*Shortest Path Problem*) y principio de subestructura óptima.
- **Algoritmo de Dijkstra** para redes con pesos no negativos (implementación con montículos binarios).
- **Algoritmo de Bellman-Ford**: tratamiento de aristas negativas y detección de ciclos absorbentes negativos.
- **Algoritmo de Floyd-Warshall**: caminos mínimos entre todos los pares de vértices mediante programación dinámica.
- Caminos en digrafos acíclicos dirigidos (DAGs) y ordenación topológica.

Sesión de laboratorio en Python:

- Implementación y análisis de eficiencia de Dijkstra frente a Bellman-Ford en NetworkX.
- Detección de oportunidades de arbitraje financiero modeladas como ciclos de coste negativo en digrafos.
- Renderizado visual de rutas geodésicas sobre redes viarias.

Objetivos de aprendizaje:

1. **Seleccionar** y **aplicar** el algoritmo de caminos mínimos óptimo en función de la topología y signos de los pesos de la red.
  2. **Modelar** problemas no espaciales (arbitraje, planificación de proyectos) como búsquedas de caminos mínimos.
- 

#### 4.2.3. SEMANA 13: Problemas de Flujo en Redes

Contenidos teóricos (Tema 8):

- Redes de transporte con capacidades: leyes de conservación de flujo y capacidad residual.
- **Problema del Flujo Máximo**: algoritmo de Ford-Fulkerson, algoritmo de Edmonds-Karp y cortes en redes.
- **Teorema del Flujo Máximo - Corte Mínimo (Max-Flow Min-Cut)**.
- **Problema del Flujo a Coste Mínimo (Min-Cost Flow)** y algoritmo del Símplex de Redes.
- Teorema de integridad de flujos.

Sesión de laboratorio en Python:

- Resolución de problemas de flujo máximo y corte mínimo con solvers de NetworkX (`maximum_flow`, `minimum_cut`).
- Modelado de redes de abastecimiento y distribución logística con costes lineales en CVXPY y NetworkX.

#### Objetivos de aprendizaje:

1. **Calcular** la capacidad máxima de transporte de una red identificando el cuello de botella o corte mínimo saturado.
2. **Formular y resolver** problemas de transbordo y distribución mediante flujos a coste mínimo.

### 4.2.4. SEMANA 14: Problemas de Emparejamiento (Matching) y Asignación

#### Contenidos teóricos (Tema 9):

- Emparejamientos en grafos: emparejamiento maximal, máximo y emparejamiento perfecto.
- Emparejamiento en grafos bipartitos: Teorema de Hall y Teorema de König.
- El **Algoritmo Húngaro** (Kuhn-Munkres) para el problema clásico de asignación de tareas.
- Emparejamiento en grafos generales no bipartitos: **Algoritmo de las Flores de Edmonds** (*Blossom Algorithm*) y contracción de ciclos impares.
- Teorema de estructura y descomposición de Gallai-Edmonds.

#### Sesión de laboratorio en Python:

- Resolución de problemas de emparejamiento bipartito y asignación ponderada en NetworkX y `scipy.optimize.linear_sum_assignment`.
- Implementación y experimentación del algoritmo de Edmonds para matching máximo general.

#### Objetivos de aprendizaje:

1. **Formular** problemas de asignación de recursos, personal y frecuencias como emparejamientos óptimos.
2. **Comprender** el mecanismo de contracción de flores (*blossoms*) para resolver emparejamientos en grafos generales en tiempo polinomial.

### 4.2.5. SEMANA 15: Rutas en Redes, TSP, VRP y Síntesis Final

#### Contenidos teóricos (Tema 10):

- Recorridos eulerianos: Teorema de Euler-Hierholzer y **Problema del Cartero Chino (CPP)** mediante eulerización por matching mínimo.
- Ciclos hamiltonianos: complejidad  $\mathcal{NP}$ -completa, condiciones necesarias (2-conexidad) y suficientes (Dirac, Ore, clausura de Bondy-Chvátal).

- **Problema del Viajante de Comercio (TSP)**: formulaciones de programación lineal entera (etapas  $x_{ijk}$ , subtours DFJ y compacta MTZ).
- Heurísticas de búsqueda local (2-Opt) y **Algoritmo de aproximación de Christofides** con cota 1.5 en grafos métricos.
- Introducción a la taxonomía y dimensiones operativas de los **Problemas de Rutas de Vehículos (VRP)**.

#### Sesión de laboratorio en Python:

- Implementación del algoritmo del Cartero Chino con NetworkX.
- Programación del modelo compacto MTZ en CVXPY y resolución con solvers MIP.
- Implementación de la heurística 2-Opt y del algoritmo de aproximación de Christofides.

#### Objetivos de aprendizaje:

1. **Diferenciar** con precisión la frontera de complejidad computacional entre problemas eulerianos (polinomiales) y hamiltonianos ( $\mathcal{NP}$ -duros).
  2. **Implementar** modelos de programación lineal entera y heurísticas con garantías formales para problemas de diseño de rutas de transporte.
-

## 5 Sistema de evaluación y criterios de calificación

El sistema de evaluación está diseñado para valorar de forma continua tanto la solidez teórica y el rigor formal como la competencia computacional en la resolución de problemas prácticos:

### 5.1. Convocatoria Ordinaria (Evaluación Continua)

La calificación final de la convocatoria ordinaria se obtiene a partir de tres componentes ponderados:

1. **Examen Parcial 1 (Bloque I - Optimización Matemática y Ciencia de Datos): 35 % de la nota final.**
  - Prueba escrita individual compuesta por cuestiones teórico-deductivas (demostraciones de convexidad, KKT, dualidad) y resolución analítica de problemas numéricos.
2. **Examen Parcial 2 (Bloque II - Optimización y Análisis de Redes): 35 % de la nota final.**
  - Prueba escrita individual compuesta por análisis de algoritmos sobre redes (Dijkstra, Ford-Fulkerson, matching, eulerización, formulaciones TSP) y complejidad.
3. **Evaluación Continua de Laboratorios y Proyectos en Python: 30 % de la nota final.**
  - Entrega periódica de cuadernos de laboratorio resueltos (15 %) y desarrollo de un proyecto práctico aplicado de modelado y optimización computacional (15 %).

*Nota de superación:* Para aprobar la asignatura por evaluación continua, el estudiante deberá obtener una calificación media ponderada igual o superior a **5.0 sobre 10**, con una nota mínima de **4.0 sobre 10** en cada uno de los dos exámenes parciales.

### 5.2. Convocatoria Extraordinaria

Los estudiantes que no hayan superado la asignatura en la convocatoria ordinaria dispondrán de una prueba extraordinaria global:

- **Examen Teórico-Práctico Final: 70 % de la nota final** (evalúa los contenidos de los Bloques I y II).
- **Evaluación de Prácticas de Laboratorio: 30 % de la nota final** (se conserva la nota obtenida durante el curso o se realiza una prueba práctica individual de programación en Python).

## 6 Bibliografía y recursos de referencia

### 6.1. Textos de referencia principal

- Bazaraa, M. S., Sherali, H. D., & Shetty, C. M. (2013). *Nonlinear Programming: Theory and Algorithms* (3rd ed.). John Wiley & Sons.
- Boyd, S., & Vandenberghe, L. (2004). *Convex Optimization*. Cambridge University Press.
- Ahuja, R. K., Magnanti, T. L., & Orlin, J. B. (1993). *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall.
- Bondy, J. A., & Murty, U. S. R. (2008). *Graph Theory*. Springer Graduate Texts in Mathematics.
- Nocedal, J., & Wright, S. J. (2006). *Numerical Optimization* (2nd ed.). Springer Series in Operations Research.

### 6.2. Textos complementarios de algoritmos y computación

- Diestel, R. (2017). *Graph Theory* (5th ed.). Springer Graduate Texts in Mathematics.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
- Taha, H. A. (2017). *Operations Research: An Introduction* (10th ed.). Pearson.
- Vanderbei, R. J. (2020). *Linear Programming: Foundations and Extensions* (5th ed.). Springer.

### 6.3. Recursos digitales y software

- **CVXPY Documentation:** Guía oficial y ejemplos de optimización convexa disciplinada (<https://www.cvxpy.org/>).
- **NetworkX Documentation:** Manual y tutoriales de análisis de redes y grafos en Python (<https://networkx.org/>).
- **SciPy Optimize Reference:** Documentación de algoritmos numéricos de optimización (<https://docs.scipy.org/doc/scipy/reference/optimize.html>).